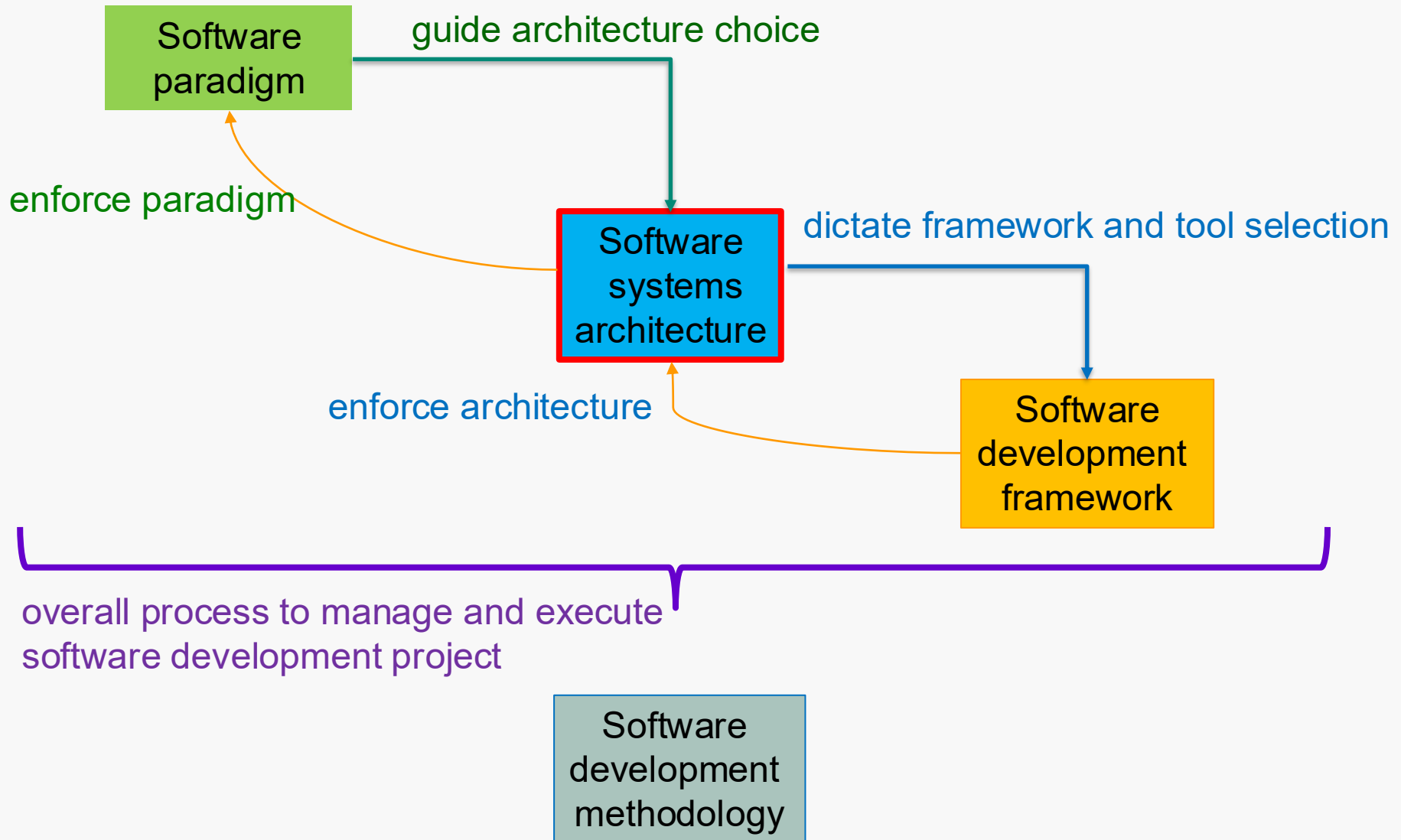


Software Engineering

- Software Systems Architecture

Dr Lily Sun

SE Conceptual Flow



Learning Objectives

- ❑ Understand Service-Oriented Architecture (SOA) as fundamentals in software system architecture
- ❑ Software architecture viewpoints, principles and decisions
- ❑ Microservice and AI-native systems evolved from SOA
- ❑ Evaluate architectural trade-offs between SOA and Microservices

What is Software Systems Architecture?

ISO/IEC/IEEE 42010:2022

❑ Fundamental organisation

- ✿ core structure of a system – functional behaviour

❑ Technical components

- ✿ major elements (i.e., modules, services, layers, data) make up a software system

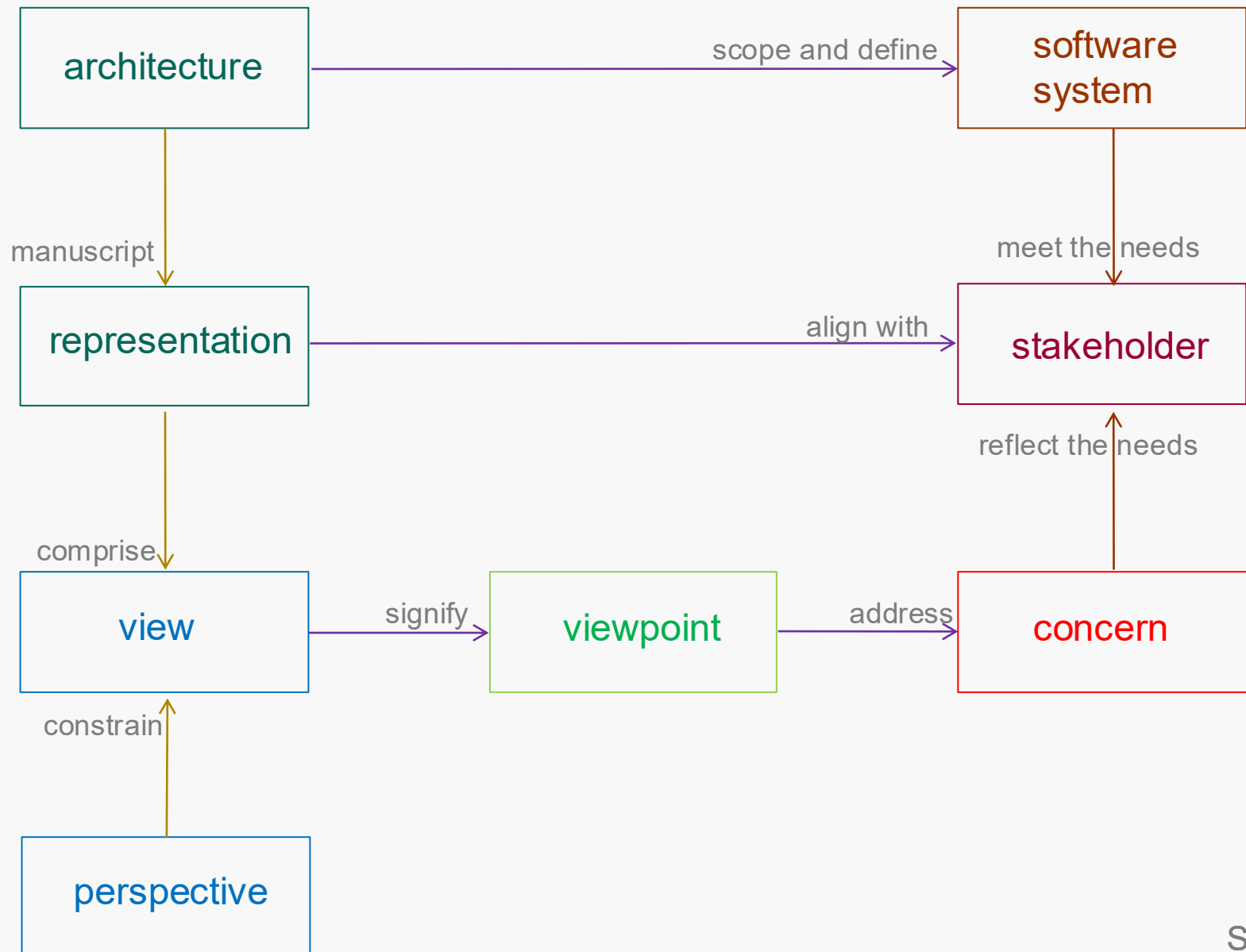
❑ Relationships (internal and environment)

- ✿ collaborative interaction amongst dependencies, interfaces, control flow (front-end to back-end) within a software system
- ✿ collaborative interaction with external systems(eco-systems), users, or devices within systems network

❑ Guiding principles for design and evolution

- ✿ architectural decisions, styles, constraints, and quality attributes (e.g., compatibility, scalability, reliability, security), ensuring the shape and sustain the system over time

Conceptual Vision for Software Architecture



Viewing Software Systems through Viewpoints



Context Viewpoint

(stakeholders, internal and external systems, dependencies and interactions between them)

Functional Viewpoint

(fundamental organisation of the system)

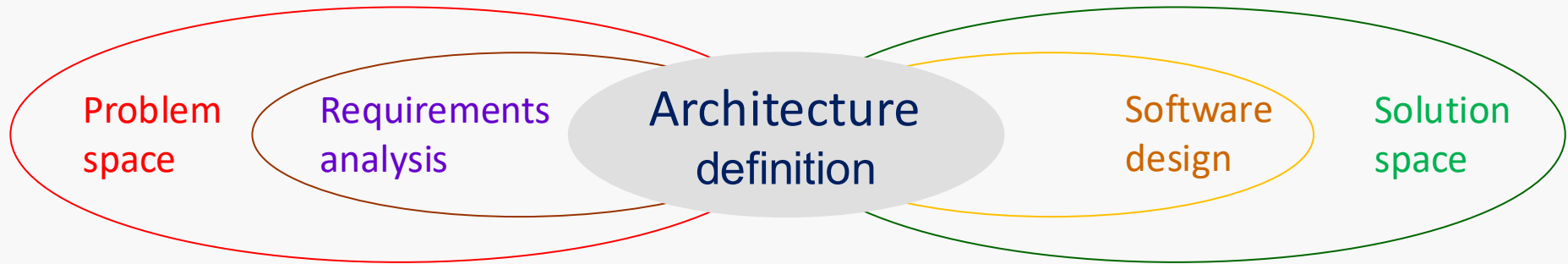
Development Viewpoint

(system construction, project management)

Deployment & Operation Viewpoint

(infrastructural environment, operational concerns)

Patterning Architectures in SE



Architecture definition is a process

- ❑ formalising stakeholders' concerns into software system requirements

and

- ❑ mapping the requirements on to software system design specifications for extensible and reusable solutions

Common Architectural Styles

focusing on high-level conceptual thinking about software systems

Style	Description	Example Use Case
Layered (n-tier)	Organizes system into layers (presentation, business, data).	Enterprise systems
Client-Server	Clients request services from servers.	Web apps
Microservices	Independently deployable services with their own data.	Cloud-native apps
Event-Driven	Components react to events asynchronously.	IoT, streaming systems
Service-Oriented (SOA)	Services communicate via standard interfaces.	Enterprise integrations
Pipe-and-Filter	Data flows through sequential transformations.	Compilers, data pipelines

Architecture Modelling Approaches

– Unified Modelling Language(UML)

focusing on visual representation of software systems – structure and behaviour

Diagram Type	Purpose	Example
Component	System's high-level building blocks	Shows services and their interfaces
Deployment	Hardware/software mapping	Servers, containers, DBs
Class	Static structure of object-oriented systems	Classes and their relationships
Sequence	Dynamic interaction flow	Order of operations between components
Use Case	External actors and system goals	Login, Order processing
Activity	Workflows and business processes	Payment workflow
State Machine	Object behavior over time	Lifecycle of an Order object

Characteristics of Original SOA

❑ Service contracts

via service register

❑ Loose coupling

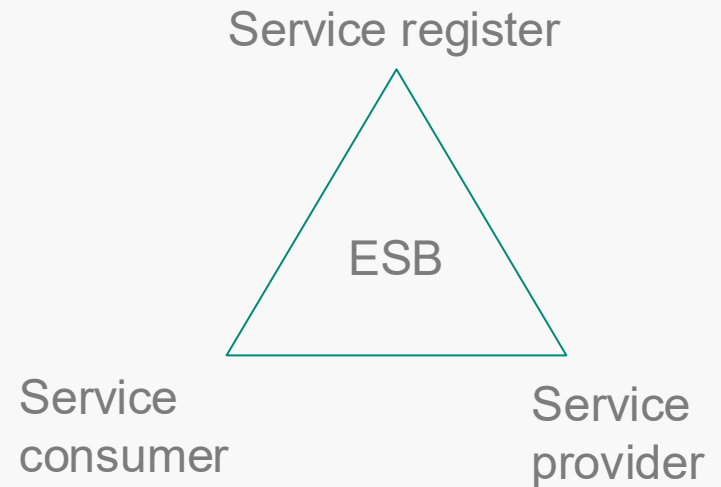
demand-driven

❑ Service composition

multiple service consumer vs
multiple service provider

❑ Network-based interaction

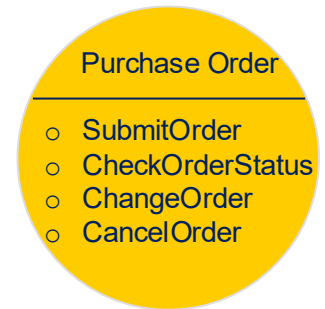
via Enterprise Service Bus



Service Concept

❑ Service

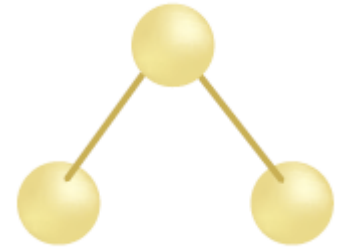
⇒ a logical unit of functionality that provides a specific capability



❑ Service compositions

⇒ coordinated aggregation of *services*

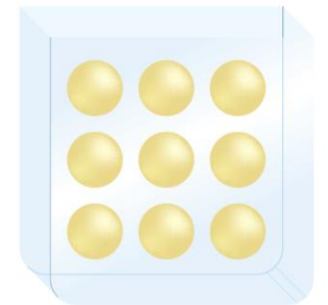
⇒ business context-aware



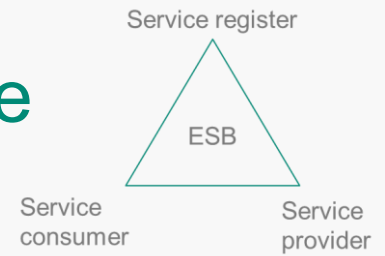
❑ Service inventory

⇒ standardise and governed container – a pool of registered services

⇒ a meaningful segment of an enterprise



Original SOA vs AI-Native Architecture



MCP (engine)

Model Context Protocol

1. Discover securely the domain context application services and data to AI models
2. Automatic tool discovery

SOAP

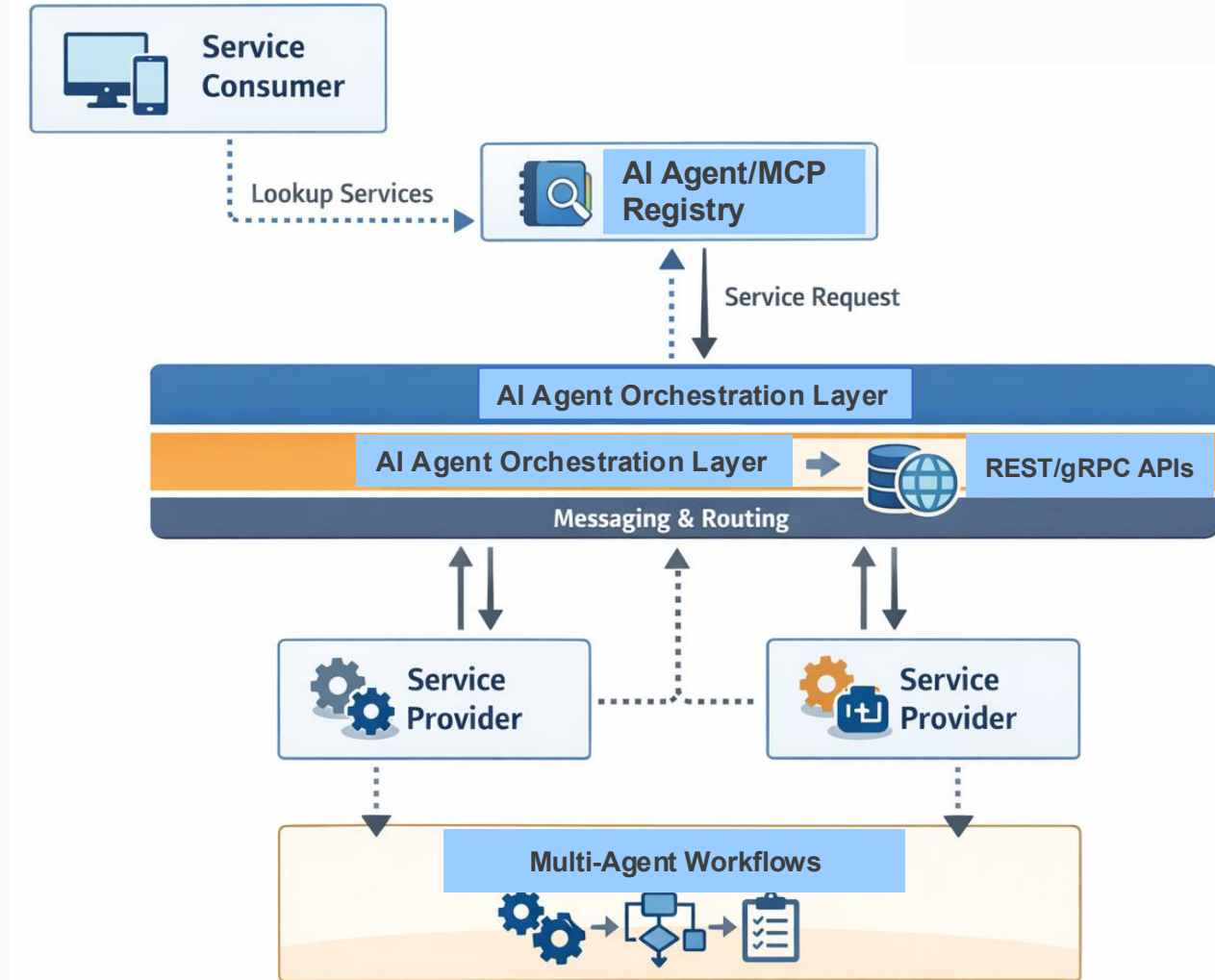
Standards-based XML messaging for requested service

1. Message encryption and digital signature
2. Banking, healthcare, insurance and government

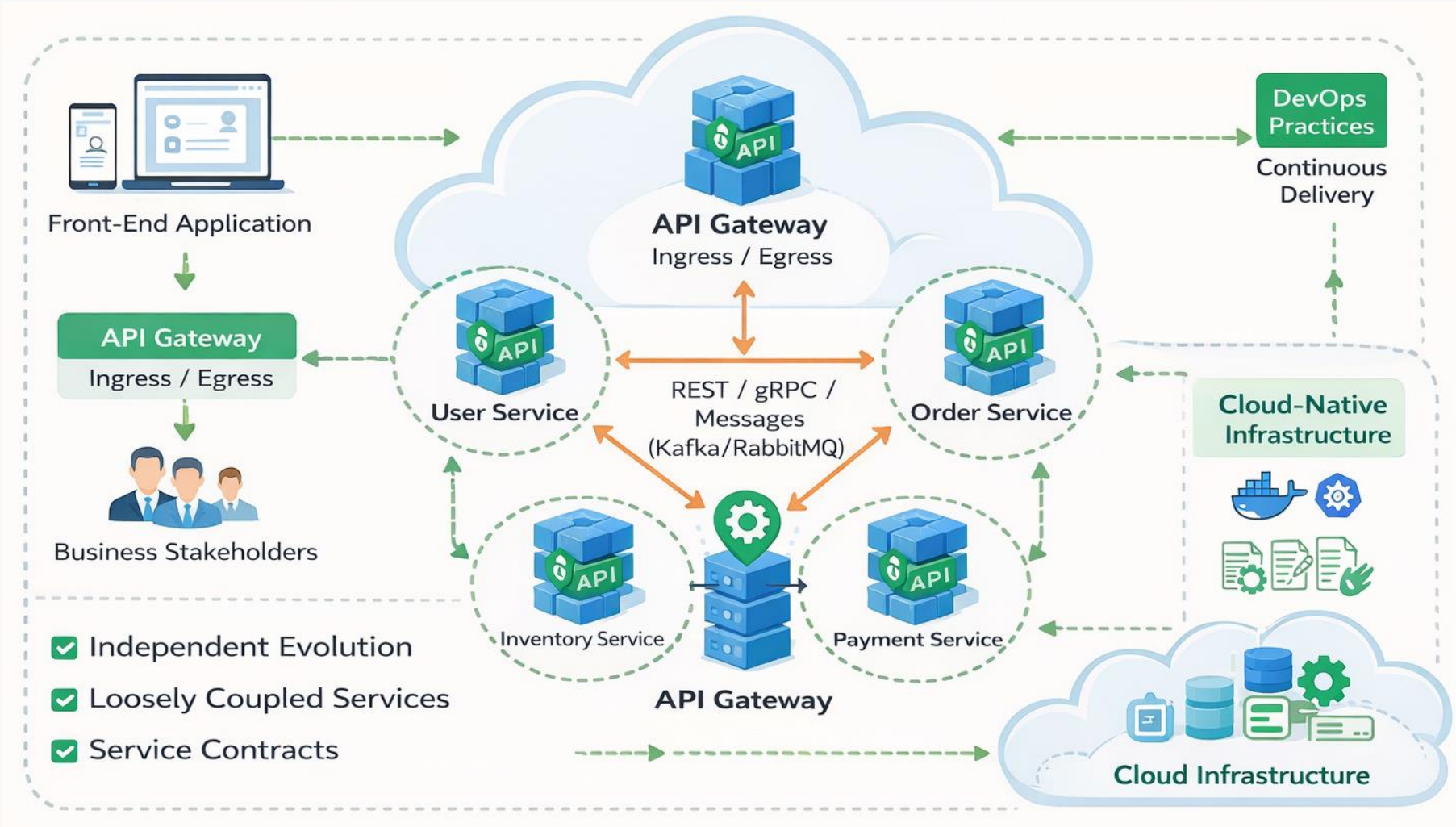
REST

Representational State Transfer

1. Performance and simplicity
2. Cloud-native and microservices architecture style
3. Manual API documentation



Modern SOA Architecture

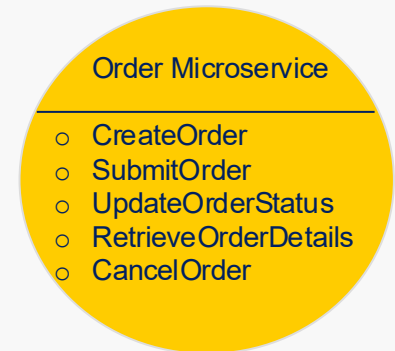


Microservices Underpinned by Modern SOA

❑ Microservices – architectural style

🦋 A microservice is a service that is independently deployable and runs as its own process

- owns a single business capability
- has its own database (or schema)
- communicates over the network (HTTP, gRPC, messaging)
- independently developed, deployed, scaled, and versioned
- failure is isolated from other microservices



Microservices Underpinned by Modern SOA (cont.)

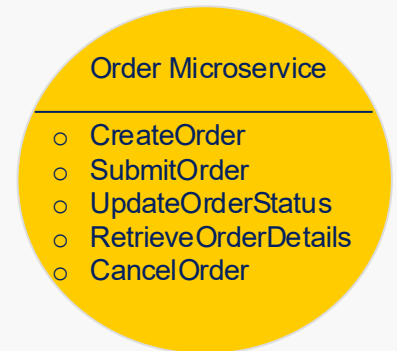
□ Microservices

🔥 Technology Independence for services

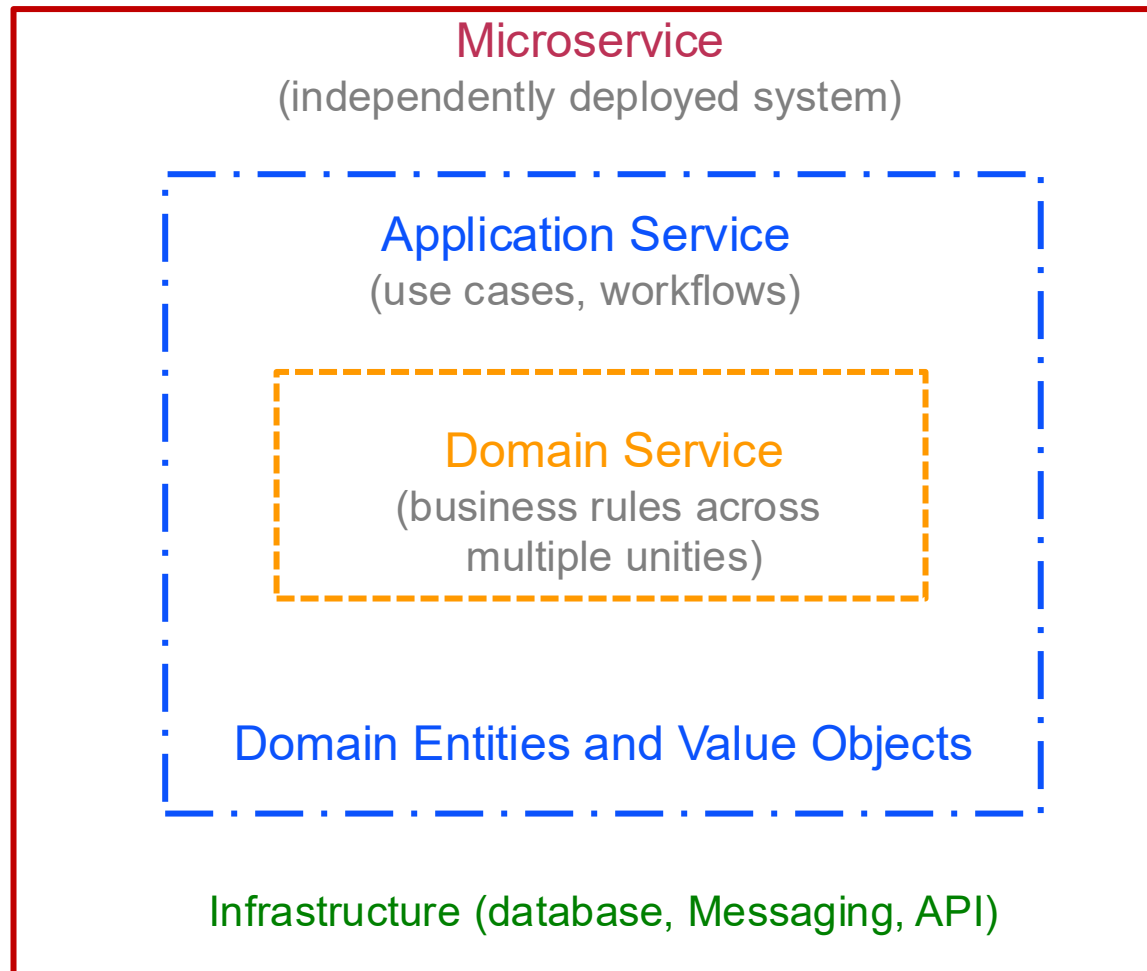
- different languages
- different frameworks
- different databases

🔥 Automated DevOps

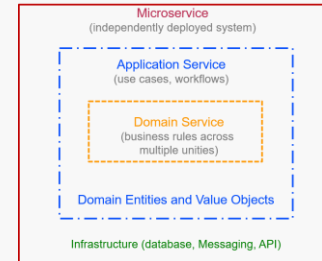
- CI/CD pipelines
- containers (Docker – containerisation platform)
- orchestration (Kubernetes – open-source container orchestration platform)
- monitoring & logging



Microservice - Architectural Abstraction



Structure of Order (Micro)Service



1. Presentation Layer (API / UI Layer)



Purpose

- expose Order Service capabilities to the business world in a standardised and technology-agnostic way



Service consumers

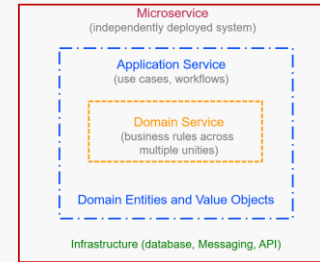
- business applications
- partner systems in the business network (eco-environment)
- other enterprise services (by contract via ESB)



Capabilities

- defines service contracts (WSDL/OpenAPI)
- exposes business operations, not implementation details, e.g. **CreateOrder**, **CancelOrder**, **RetrieveOrderDetails**
- handles input/output schemas (XML/JSON)
- enforces validation, security, and access policies
- provides loose coupling between consumers and service logic

Structure of Order (Micro)Service (cont.)



1. Presentation Layer (API / UI Layer)

🦁 Capabilities

🌱 defines service contracts in OpenAPI

Endpoints (paths, /users, /orders)

HTTP methods

Request schema (parameters and message body)

Response schema (status codes {success cases / client errors / server errors}, data schema)

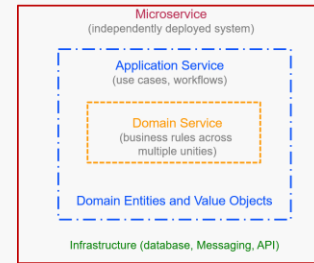
Authentication (Security Schemes in OpenAPI document)

🌱 exposes business operations, not implementation details

OrderController

```
____ createOrder()
____ retrieveOrder()
____ cancelOrder()
```

Structure of Order (Micro)Service (cont.)



1. Presentation Layer (API / UI Layer)

🏰 Capabilities

- REST controller: manage input/output schemas via REST (Representational State Transfer) and HTTP methods

- REST controller

Resource URL (location)

HTTP methods

JSON or XML*

Status codes

- HTTP methods

POST /orders

GET /orders/{id}

PUT /orders/{id}/cancel

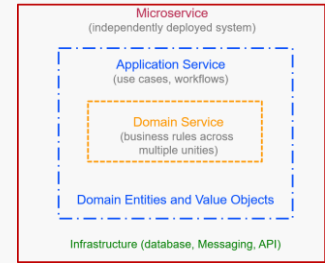
DELET /order{id}

*XSD defines XML structure for validation

- Business logic: validation, rules, orchestration

- Data access: repositories, ORM (Object-Relational Mapping) / SQL

Structure of Order (Micro)Service (cont.)



2. Application Layer (Service Layer / Business Logic Layer)

the orchestrator of use cases in a microservice



Use Case Coordination

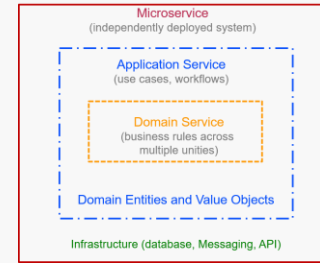
- Implements application-specific workflows (e.g., Place Order, Approve Payment)
- Calls domain objects and services in the correct sequence
- Ensures that business rules are applied consistently across requests



Service Interfaces

- Exposes operations that the presentation layer (API/controllers) can invoke
- Defines clear service boundaries and contracts
- Often implemented as application services or use-case handlers

Structure of Order (Micro)Service (cont.)



2. Application Layer (Service Layer / Business Logic Layer)

the orchestrator of use cases in a microservice

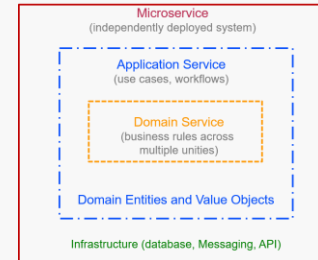
Transaction Management

- Controls transactional boundaries across multiple domain operations
- Ensures data consistency within a single microservice
- Coordinates commits or rollbacks when necessary

Authorization and Validation

- Enforces application-level authorization rules (who can perform which action)
- Performs request validation that goes beyond simple input checks
- Delegates pure business rule validation to the domain layer

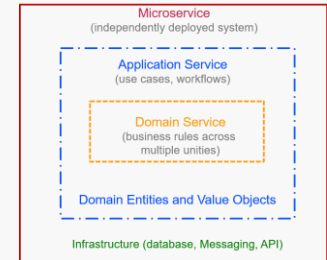
Structure of Order (Micro)Service (cont.)



2. Application Layer (Service Layer / Business Logic Layer) the orchestrator of use cases in a microservice

- ✚ Integration with Infrastructure via Abstractions
 - Uses interfaces or ports to interact with databases, message brokers, and external services
 - Remains independent of specific technologies or frameworks
 - Supports easier testing by allowing mocks or stubs.

Structure of Order (Micro)Service (cont.)



3. Domain Layer (Domain / Model Layer)

the heart of the microservice – domain context

🌿 Domain Entities

- Represent core business objects (e.g., Order, Customer, Payment)
- Contain state and behaviour, not just data
- Enforce business invariants (rules that must always hold true)

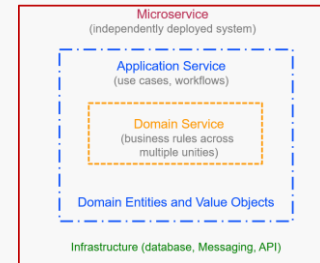
🌿 Value Objects

- Model immutable concepts such as `Money`, `Address`, or `Email`
- Defined by their values rather than identity
- Improve expressiveness and reduce errors in business logic

Structure of Order (Micro)Service (cont.)

3. Domain Layer (Domain / Model Layer)

the heart of the microservice



Domain Services

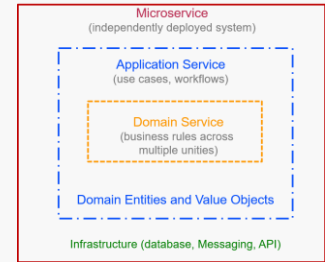
- Implement business logic that does not naturally belong to a single service (e.g., an order can be confirmed only if customer is still valid and payment is successful)
- Coordinate multiple entities to fulfil a domain norm
- Remain free of application or infrastructure concerns, only on business rules



Aggregates and Boundaries

- Group related entities and value objects into aggregates
- Define consistency boundaries and rules for state changes
- Ensure that modifications occur through a controlled aggregate root

Structure of Order (Micro)Service (cont.)



3. Domain Layer (Domain / Model Layer)

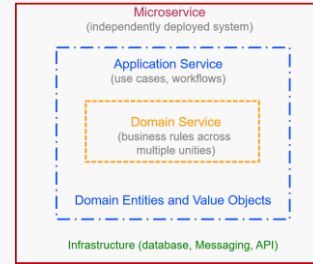
the heart of the microservice



Domain Events

- Capture significant business occurrences (e.g., OrderPlaced, PaymentFailed)
- Enable decoupled communication within or across services
- Represent facts that have already happened in the domain

Structure of Order (Micro)Service (cont.)



4. Infrastructure Layer (Data Access / Integration Layer)

all technical and operational concerns

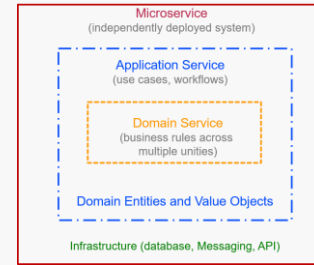
🌿 Database Access

- Implements repositories or data access objects (DAOs)
- Manages CRUD operations, transactions, ORM mappings, and query optimization
- Abstracts the underlying database technology (SQL, NoSQL, in-memory, etc.) from the rest of the service

🌿 Messaging and Event Handling

- Integrates with message brokers (e.g., AWS SQS)
- Publishes and consumes events or messages for asynchronous communication
- Ensures reliability, retries, and message serialization / deserialization

Structure of Order (Micro)Service (cont.)



4. Infrastructure Layer (Data Access / Integration Layer)

all technical and operational concerns

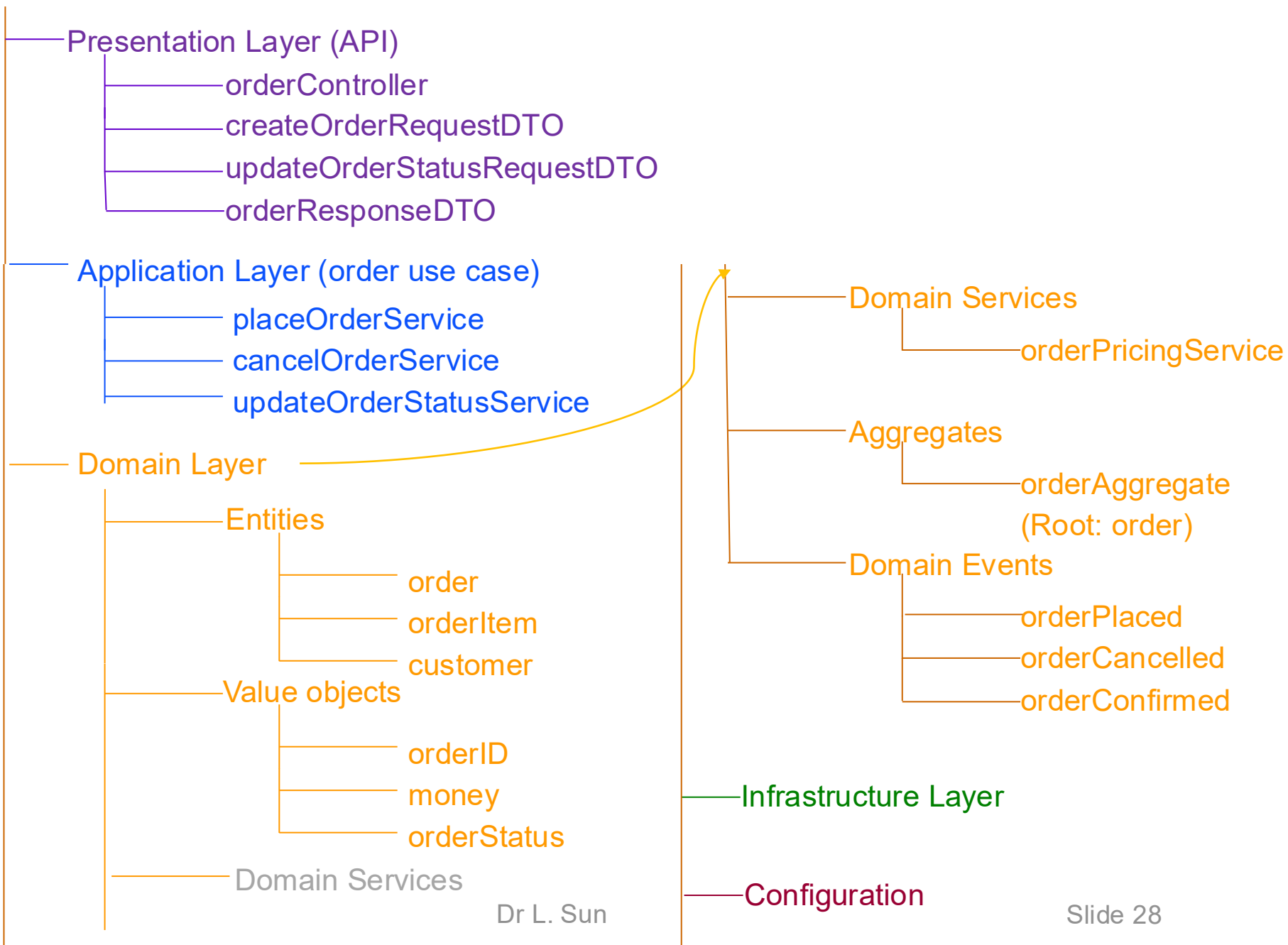
🌿 External Service Integrations

- Communicates with other microservices or third-party systems via REST, gRPC, SOAP, or SDKs
- Handles API clients, authentication tokens, timeouts, retries, and circuit breakers
- Shields the application layer from changes in external APIs

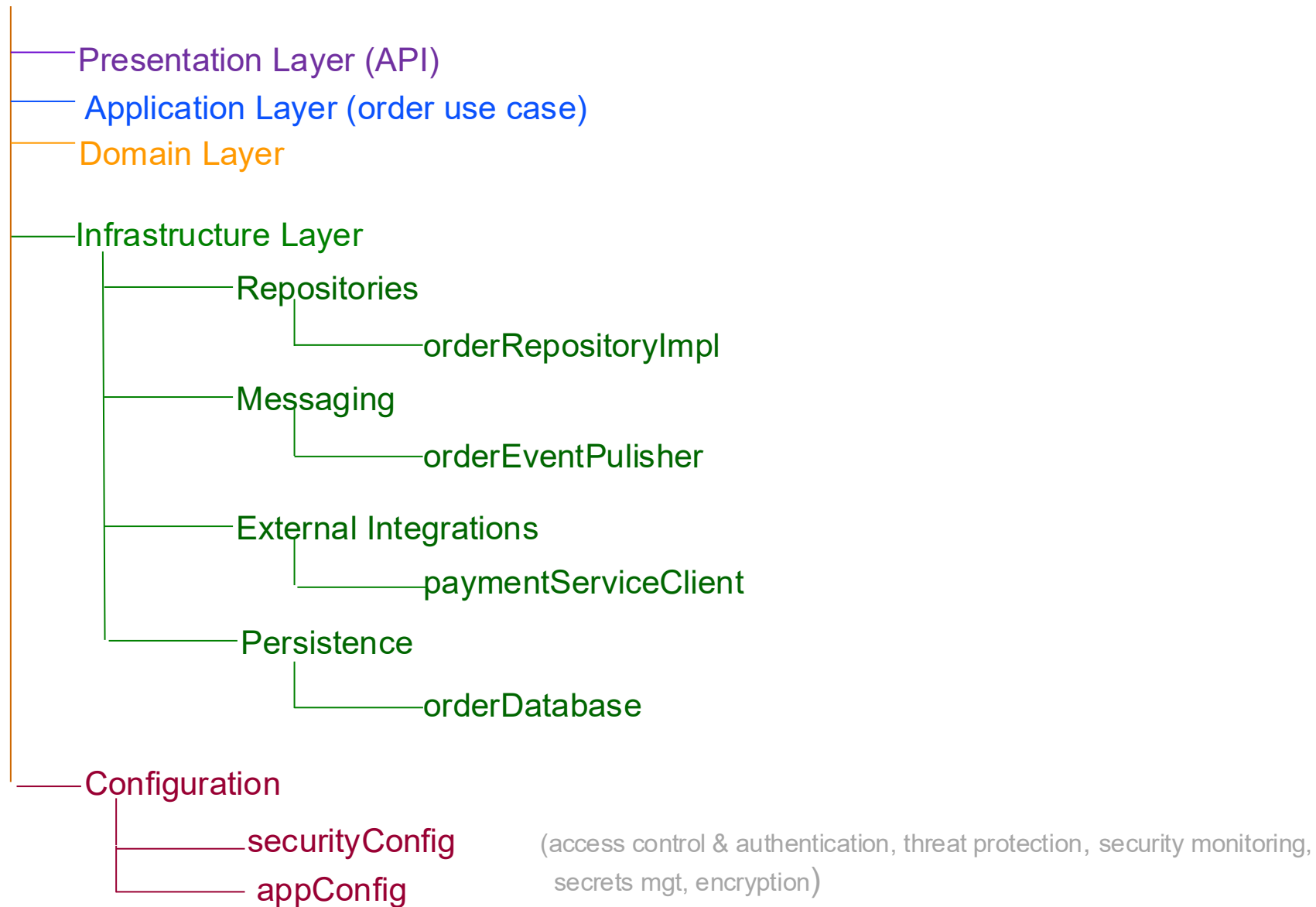
🌿 Technical Cross-Cutting Concerns

- Logging, monitoring, and tracing
- Security implementations such as encryption, credential storage and secure connections
- Configuration management and environment-specific settings

Order Microservice



Order Microservice



Service vs Microservice

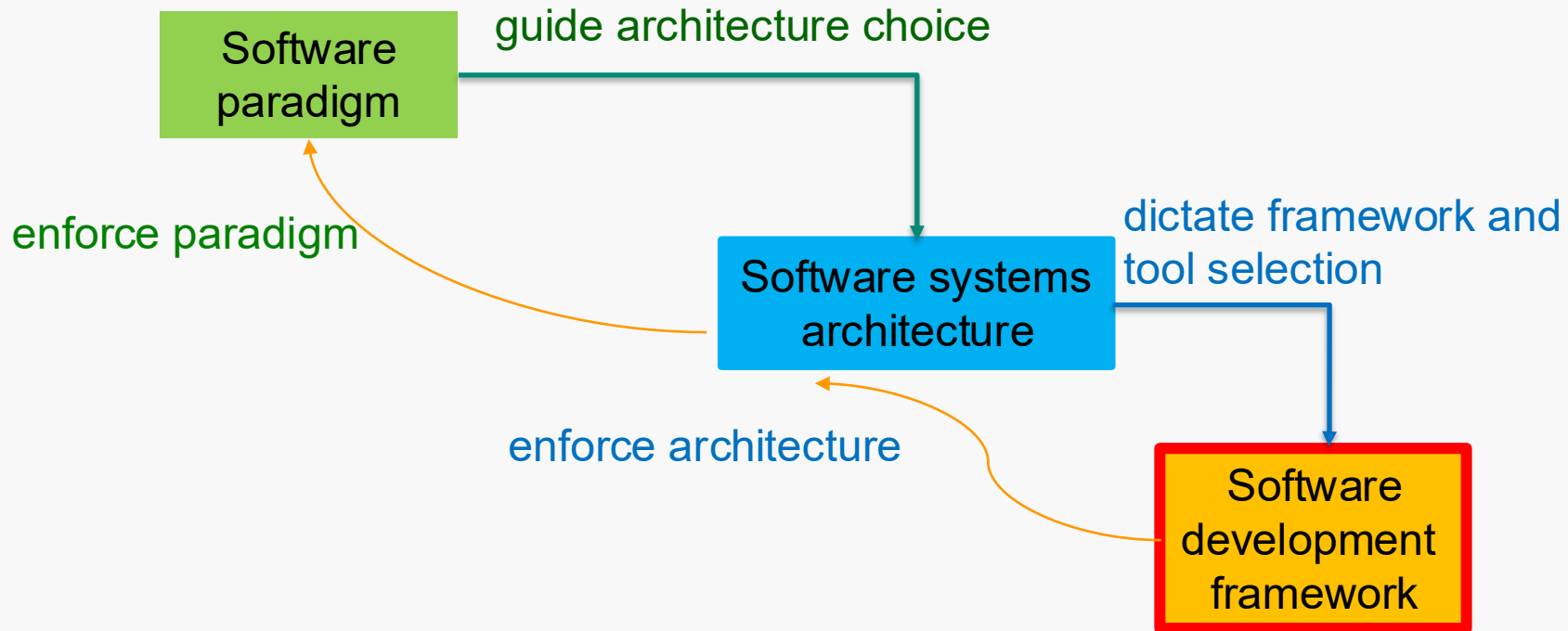
Aspect	Service	Microservice
Meaning	Logical unit of functionality	Independently deployable service
Deployment	Shared	Independent
Runtime	Same process	Separate process
Database	Often shared	Owens its data
Coupling	Often tight	Loosely coupled
Scope	Broad term	Specific architectural pattern

All microservices are services.
Not all services are microservices.

Service (Application or Domain) vs Microservice

Aspect	Microservice	Application Service	Domain Service
Level	Architecture	Application design	Domain design
Deployment	Independent	Not deployable	Not deployable
Purpose	Business capability	Use case orchestration	Business rules
Technology-aware	Yes	Some	No
Talks to DB/API	Yes (via infra)	Yes (via interfaces)	No
Knows infrastructure	Yes	Indirectly	Never

SE Conceptual Flow



overall process to manage and execute software development project

Software development methodology